Architecture Document

SledSurfers — Unity 6 C# Project

Version 1.0 — March 2026

Covers: Architecture Decisions, Dependency Injection, Design Patterns, SDK & Library Choices, Project Structure, and C# Coding Standards

1. Project Overview

SledSurfers is a sled-based endless runner built in Unity 6. The player launches via a slingshot mechanic, steers downhill collecting coins and avoiding obstacles, then spends coins on upgrades between runs. The codebase is structured around clean separation of concerns, interface-driven dependency injection, and event-based communication.

The project comprises 56 C# source files (~3,550 lines) organized across 7 domain folders. No file exceeds 220 lines. All gameplay, UI, and service logic flows through abstractions registered in VContainer lifetime scopes.

2. SDK & Library Choices

The following table lists every external dependency with the reasoning behind each choice.

Dependency	Source	Purpose	Trade-off / Reasoning
Unity 6 (6000.x)	Unity Technologies	Game engine, rendering (URP), physics, scene management	Unity 6 provides Awaitable, improved async support, and linearVelocity API on Rigidbody. Chosen over 2022 LTS for long-term support.
Universal Render Pipeline	Unity built-in	Lightweight rendering, camera stacking for UI overlay	URP is lighter than HDRP, suitable for mobile targets. Camera stacking enables a dedicated UI camera on top of gameplay.
VContainer	GitHub / OpenUPM	Dependency Injection framework	Chosen over Zenject for lighter footprint, faster resolve times, and Unity 6 compatibility. Pure C# with no code generation.
UniTask (Cysharp)	GitHub / OpenUPM	Zero-allocation async/await	Used in BootstrapFlow for scene loading. Avoids Task.Yield() GC overhead. Integrates with Unity PlayerLoop.
Unity Input System	com.unity.inputsystem	Player input abstraction	InputHandler wraps PlayerInputActions. SimpleInputHandler provides legacy Input.GetAxis fallback. Both implement IInputHandler, swappable via DI.
TextMeshPro	Unity built-in	UI text rendering	Standard for all in-game text. Better typography and performance than legacy UI.Text.
Unity Physics	Unity built-in	Rigidbody, collisions, triggers	Uses Rigidbody.linearVelocity (Unity 6 API), continuous collision detection, interpolation. PlayerMotor is the single physics contact point.

3. Architecture Decisions

Each decision is documented with reasoning and acknowledged trade-offs.

3.1 VContainer over Zenject

Decision: Use VContainer as the DI framework.

Reasoning: VContainer is ~5x faster at resolve time than Zenject in benchmarks, has no dependency on code generation, and is actively maintained for Unity 6. Its LifetimeScope component aligns naturally with Unity's scene lifecycle. Constructor injection for plain C# and [Inject] method injection for MonoBehaviours cover all use cases.

Trade-off: Smaller community and fewer tutorials than Zenject. Mitigated by thorough DI comments and this document.

3.2 Three-Tier Scope Hierarchy

Decision: Organize DI into three lifetime scopes: GameLifetimeScope (root), GameplayLifetimeScope (gameplay scene), UILifetimeScope (UI scene).

Reasoning: GameLifetimeScope is DontDestroyOnLoad and holds singletons persisting across scene loads. GameplayLifetimeScope is a child scope living with the Gameplay scene. UILifetimeScope is a child scope in the additive UI scene, inheriting services from the root. This mirrors the scene structure.

Trade-off: Child scopes add complexity vs. a single flat container. Justified because scene-scoped lifetimes prevent stale references and match Unity's scene-based memory model.

3.3 Event-Driven Communication

Decision: Use C# events and delegates for inter-system communication (OnSpeedChanged, OnCrash, OnCoinCollected, OnStateChanged, OnRunEnded, etc.).

Reasoning: Events provide compile-time type safety, zero per-frame cost when nothing changes, and clear subscription/unsubscription points. Avoids the overhead and debugging difficulty of a global message bus.

Trade-off: Events require manual unsubscription to prevent leaks. Mitigated by consistent Unsubscribe() in MonoBehaviours and IDisposable on plain C# classes.

3.4 GameplayFlow / RunSession Split

Decision: Split gameplay orchestration into GameplayFlow (menu navigation + session lifecycle) and RunSession (active-run loop: launch, charge, run, score, end).

Reasoning: The original GameplayFlow was 286 lines handling two concerns. After the split, each is ~160 lines with a single reason to change. RunSession is created per-run as a plain C# class (not a VContainer entry point) because its lifetime is shorter than the scene.

Trade-off: One more class in the graph. Justified because each class is independently testable with clear boundaries.

3.5 Dual State Machine

Decision: Use a global GameState enum (via IGameStateManager) and local FlowPhase / RunSession.Phase enums.

Reasoning: GameState drives cross-cutting concerns: UIService listens and fires screen show/hide events. The UI layer never needs to know about launch phases. FlowPhase and Phase are gameplay implementation details. Separating them prevents UI from coupling to gameplay internals.

Trade-off: Two state machines to reason about. Mitigated by operating at different granularities with no overlap.

3.6 Additive Scene Loading for UI

Decision: Load the UI scene additively with a dedicated UI camera stacked via URP camera stacking.

Reasoning: Separates UI hierarchy from gameplay completely. Designers edit UI and gameplay scenes independently without merge conflicts. CameraStackHandler manages adding/removing the UI camera at runtime.

Trade-off: Camera stacking adds small rendering overhead. Additive scenes require careful lifecycle management. Both are well-understood Unity patterns.

3.7 PlayerPrefs for Persistence

Decision: Use PlayerPrefs via PlayerPrefsDataService implementing IPlayerDataService.

Reasoning: Simplest possible persistence for a prototype. The IPlayerDataService interface allows swapping to file-based or cloud persistence without modifying any consumer.

Trade-off: PlayerPrefs is not secure and has platform size limits. The interface abstraction makes migration trivial.

3.8 Initialize() Wires, Start() Triggers

Decision: MonoBehaviour [Inject] methods only store references. Side effects are triggered by IStartable.Start() or the orchestrator.

Reasoning: VContainer does not guarantee [Inject] order across RegisterComponent calls. Level managers that spawned inside Initialize() failed on first run because pools were not yet ready. Moving spawning to GameplayFlow.Start() eliminates the race condition.

Trade-off: Requires the orchestrator to explicitly call Reset() for initial spawn. This is better design: initialization should wire dependencies, not produce side effects.

4. Project Structure

Folder	Responsibility	Key Files
Scripts/Core/DI	VContainer composition roots	GameLifetimeScope, GameplayLifetimeScope
Scripts/Core/Interfaces	All abstractions (DIP contracts)	IPlayerMotor, IInputHandler, IGameStateManager, IUIService, ILevelManager, ICoinDespawner
Scripts/Core/Services	Concrete service implementations	GameStateManager, PlayerPrefsDataService, UpgradeService, PoolManager, SceneLoaderService
Scripts/Core/SceneManagement	Bootstrap and scene loading	BootstrapFlow
Scripts/Data/Models	Pure data classes (no behavior)	PlayerData
Scripts/Data/ScriptableObjects	Designer-editable configuration	GameSettings, SpawnConfig
Scripts/Gameplay	Active gameplay logic	GameplayFlow, RunSession, CoinLevelManager, ObstacleLevelManager
Scripts/Gameplay/Player	Player subsystems	PlayerManager, PlayerMotor, PlayerPhysicsController, PlayerCollisionHandler, MomentumTracker
Scripts/Gameplay/Slingshot	Launch mechanics	SlingshotController
Scripts/Gameplay/Camera	Follow camera	PlayerCameraController
Scripts/Pooling	Object pool system	IPoolManager, PoolManager, PoolConfig, LevelPoolProvider
Scripts/UI	UI screens and service	UIService, IUIService, StartMenuScreen, HUDScreen, GameOverScreen
Scripts/UpgradeSystem	Upgrade logic and UI	UpgradeService, UpgradeConfig, UpgradeScreen, UpgradeItemView
Scripts/Utils	Utilities	CameraStackHandler

5. Dependency Flow

All dependencies flow downward through interfaces. No concrete class is injected directly except MonoBehaviour components (required by VContainer's RegisterComponent).

5.1 GameLifetimeScope (Root — Singleton)

Registration	Interface	Lifetime
GameSettings	(instance)	Singleton
UpgradeConfig	(instance)	Singleton
SceneLoaderService	ISceneLoader	Singleton
GameStateManager	IGameStateManager	Singleton
PlayerPrefsDataService	IPlayerDataService	Singleton
ScriptableObjectUpgradeConfigProvider	IUpgradeConfigProvider	Singleton
UpgradeService	IUpgradeService	Singleton
PoolManager	IPoolManager	Singleton
UIService	IUIService	Singleton
BootstrapFlow	(entry point)	Singleton

5.2 GameplayLifetimeScope (Child — Scene-Scoped)

Registration	Interface(s)	Lifetime
SimpleInputHandler	IInputHandler	Scoped
PlayerManager	(component)	Scoped
LevelPoolProvider	(component)	Scoped
ObstacleLevelManager	ISpawnPositionProvider, ILevelManager	Scoped
CoinLevelManager	ICoinDespawner, ILevelManager	Scoped
GameplayFlow	(entry point)	Scoped

5.3 UILifetimeScope (Child — Additive Scene)

Registers screen MonoBehaviours as components. All screens inject IUIService from the parent scope. No new services are registered here.

6. Design Patterns

Pattern	Where Used	Purpose
Composition Root	GameLifetimeScope, GameplayLifetimeScope, UILifetimeScope	Single place where all dependencies are wired. No new() calls scattered through gameplay code.
Observer	C# events (OnSpeedChanged, OnCrash, OnRunEnded, OnStateChanged, etc.)	Decoupled communication. Camera, UI, MomentumTracker all react to motor events independently.
Facade	PlayerManager	Simplifies access to Motor, Slingshot, MomentumTracker, CollisionHandler behind one MonoBehaviour.
Object Pool	PoolManager + LevelPoolProvider + PoolConfig	Avoids runtime allocation for coins/obstacles. Config-driven via ScriptableObject.
State Machine	GameState (global), FlowPhase (GameplayFlow), Phase (RunSession)	Governs transitions at appropriate granularities.
Mediator	UIService	Translates GameState changes into screen-specific events. Neither screens nor gameplay know about each other.
Strategy	IInputHandler (SimpleInputHandler / InputHandler)	Swappable input implementations via DI.
DTO	PlayerData, GameStateData, UpgradeItemData	Pure data carriers between layers. No behavior, no dependencies.

7. SOLID Principles Application

7.1 Single Responsibility

Every class has one reason to change. PlayerMotor handles physics. SlingshotController handles charge mechanics. RunSession handles the active-run loop. GameplayFlow handles navigation. UIService handles state-to-screen mediation.

7.2 Open/Closed

UpgradeService uses dictionary-based dispatch (LevelGetters / LevelSetters) so new upgrade types require adding entries, not editing methods. Level managers implement ILevelManager; adding a new spawnable type requires a new class and DI registration — no existing code changes.

7.3 Liskov Substitution

SimpleInputHandler and InputHandler are fully interchangeable through IInputHandler. PlayerPrefsDataService can be replaced with any IPlayerDataService. No implementation has surprising behavior.

7.4 Interface Segregation

Interfaces are focused: ICoinDespawner (1 method), ILevelManager (1 method), ICollisionHandler (2 events), IInputHandler (3 properties + 2 methods). IUIService is the largest but justified by its mediator

role.

7.5 Dependency Inversion

All service-to-service dependencies flow through interfaces in Core/Interfaces. No FindFirstObjectByType calls. UI screens inject IUIService, not UIService. PlayerCollisionHandler depends on ICoinDespawner, not CoinLevelManager. GameplayFlow depends on IReadOnlyList<ILevelManager>, not concrete managers.

8. C# Coding Standards

8.1 Naming Conventions

Element	Convention	Example
Namespace	PascalCase, mirrors folder path	SledSurfers.Gameplay.Player
Class / Struct / Enum	PascalCase	PlayerMotor, GameState, UpgradeStatType
Interface	I-prefix + PascalCase	IPlayerMotor, IUIService, ICoinDespawner
Public method / property	PascalCase	CurrentSpeed, StartTracking(), ApplyUpgrade()
Private field	_camelCase (underscore prefix)	_rigidbody, _isInitialized, _coinsCollected
Local variable / parameter	camelCase	playerData, currentLevel, spawned
Constant / static readonly	PascalCase	StorageKey, ObstacleTag, CoinTag
[SerializeField]	_camelCase (private backing)	[SerializeField] private float _height
Event	On-prefix + PascalCase	OnSpeedChanged, OnCrash, OnRunEnded
Enum value	PascalCase	GameState.Playing, PoolObjectType.Coin
Boolean	Is/Has/Can prefix preferred	IsMoving, IsCharging, CanAfford

8.2 File & Class Organization

- One class per file. File name matches class name exactly.
- Namespaces mirror folder structure (SledSurfers.Core.Interfaces for Scripts/Core/Interfaces/).
- Interfaces live in Core/Interfaces/ or in the domain folder if domain-specific (ISpawnPositionProvider in Gameplay/).
- No file exceeds 220 lines. If a class grows beyond ~200 lines, evaluate SRP and extract.

8.3 Access Modifiers

- Default to the most restrictive access. Private fields, internal classes where possible.
- All MonoBehaviour fields are private with [SerializeField] — never public fields on MonoBehaviours.
- Public API uses properties (get-only where possible). Exception: PlayerData uses public fields for JsonUtility compatibility.

8.4 Dependency Injection Rules

- Plain C# classes use constructor injection exclusively. No [Inject] on non-MonoBehaviours.
- MonoBehaviours use [Inject] public void Construct(...) or Initialize(...) for method injection.
- [Inject] methods must only store references. No side effects. Behavior is triggered by IStartable.Start() or orchestrator calls.
- Register implementations against interfaces (.As<IFoo>()). Never .AsSelf() unless strictly necessary (avoided in this project).

8.5 Event Subscription Rules

- Every Subscribe() must have a matching Unsubscribe() in OnDestroy() or Dispose().
- Classes subscribing to injected-dependency events must implement IDisposable.
- Handlers are private methods named Handle[EventName] (e.g., HandleCrash, HandleSpeedChanged).
- Never use anonymous lambdas for event subscriptions — they cannot be unsubscribed.

8.6 MonoBehaviour Rules

- Use [RequireComponent] to declare component dependencies.
- Prefer GetComponent<T>() in Initialize/Awake for sibling components. Use [SerializeField] for cross-object references.
- Never use FindFirstObjectByType or any scene-scanning API. All dependencies come through DI.
- Never use static singletons. All shared state flows through VContainer services.

8.7 XML Documentation

- All interfaces must have XML doc comments on the interface and non-obvious members.
- Public architectural-boundary classes should have a class-level summary with design rationale.
- Trade-off comments ("Trade-off: ...") are encouraged on decisions a future developer might question.

8.8 Formatting

- Allman-style braces (opening brace on its own line).
- 4-space indentation (no tabs).
- One blank line between methods. Two blank lines between sections (fields vs. methods).
- No regions. Organize: fields, constructor/Initialize, public API, private methods, Unity lifecycle, editor-only.
- using directives: System first, Unity second, project third, third-party last.

8.9 Logging

- All Debug.Log calls use a [ClassName] prefix tag (e.g., Debug.Log("[PlayerMotor] Launched")).
- For production, gate behind a log-level system or [Conditional("DEBUG")]. Known TODO.

8.10 Async / Await

- Use UniTask for async operations in Unity context. Avoid System.Threading.Tasks.Task.
- Use .Forget() for fire-and-forget UniTask calls (BootstrapFlow.Start() pattern).
- SceneLoaderService currently uses Task — should migrate to UniTask. Known TODO.

9. Known Limitations & Future Work

• PoolManager.Return() uses Queue.Contains() for duplicate detection — O(n) per call. Should use a HashSet for O(1).

• SlingshotController._maxChargeTime is hardcoded at 1.5f. Should move to GameSettings.

• MomentumTracker has unused _stoppedAtTime and _stallDuration fields — incomplete stall-timer logic. Implement or remove.

• IUpgradeConfigProvider.LoadAsync() returns Task.CompletedTask in the only implementation. Over-specified but retained for future remote loading.

• SceneLoaderService uses System.Threading.Tasks instead of UniTask. Should migrate.

• No unit tests yet. UpgradeService, GameStateManager, MomentumTracker, and RunSession are trivially testable pure C# classes.

• Debug.Log statements should be gated behind a log-level system before production.

• PlayerData uses public fields and manual Clone(). Adding a field requires updating Clone() — consider code generation.